Peak-Hour Boundary Detection: When Does Rush Hour Really Start?

Aryan, Eugene, Hans, Necmi, Saurabh, Van

University of Waikato, Hamilton, New Zealand

Abstract

Urban transit scheduling operates on static peak-hour windows (e.g., 06:00–09:00) which assume a stationary demand behavior, whereas the **real** demand for public transit may peak before or after the scheduled window and even shift over time as a result of seasonal conditions (weather, public holidays, etc.) If the schedule is misaligned, buses might arrive past the time of peak demand, leading to queue overcrowding and downstream network congestion. In this paper, we implement an online, prequential stream-mining methodology and treat the Salvador Urban Network for Transport (SUNT) dataset as an infinite data stream, with a resolution of 5-minute boarding aggregations, in order to extract concept drifts caused by empirical boarding demand, as well as structural regime/distribution shift boundaries. These results will then be compared with the static General Transit Feed Specification (GTFS) tables, alongside a naive scheduling baseline, in order to find actionable routes – tackling both a problem of (stream-inherent) demand drift, and one of (schedule-alignment) operational efficacy.

Keywords: Data Streams, Concept Drift, Public Transportation, Adaptive Random Forest, ADWIN

1. Introduction

Given the volume of boarding passengers $V_{x,t}$ at a specific transit stop x and time t, the real demand boundary appears when the rate of change of $V_{x,t}$ crosses a behavioral threshold. This demand behavior is different from the transport capacity provided by the General Transit Feed Specification (GTFS) schedule, which is a static data format that requires regular updates and is deployed days/weeks in advance. Our analysis studies the behavior of the non-stationary boarding volume of passengers and its relationship to the *inelastic* transport schedules of the transit network of Salvador, Brazil.

To ground the theory, *platform dynamics* can be formalized as a mass-balanced, fluid queueing system – different from most applications which treat the transit flow as the fluid. Let $\rho(t)$ be a given platform's (x) passenger arrival rate extracted from the boarding stream $\mathcal{S}_x(t)$, and $\kappa(t)$ the scheduled bus capacity (acting as drainage) from the static schedule table $\mathcal{T}_x(t)$. The queue accumulation function at some node x is given by

$$Q(t) = \int_{t_0}^{t} (\rho(\tau) - \kappa(\tau))d\tau \tag{1}$$

When scheduled stops lag behind the real demand peak ($t_{\text{stop}} > t_{\text{peak}}$), especially during moments of demand drift, the network enters a critical state for time $t \in [t_{\text{peak}}, t_{\text{stop}}]$. In this scenario, the corresponding equation uses an active stream $\rho(t) > 0$, while the system drainage stays null ($\kappa(t) = 0$) since no bus arrival from $\mathcal{T}_x(t)$ has yet been observed. This forces $\frac{d}{dt}Q(t) = \rho(t) > 0$, causing $Q(t)$ to grow monotonically. The capacity lag leads to platform overcrowding and a bottleneck that disrupts the entire downstream process of the network after x. We say this represents an *operational* divergence $\mathcal{S}$ and $\mathcal{T}$. Since real concept drifts alter $\mathcal{S}$ in non-predictable ways (particularly for $\mathcal{T}$), this divergence is also dependent on the existence of any drifts.

SUNT's operational framework relies on tables such as `stops.txt`, `calendar.txt`, and `stop_times.txt`, suggesting that the official scheduling is static and batch-compiled. This file format allows for the capture of predictable variance (e.g., smaller $V_{x,t}$ on weekends) but remains blind to stochastic boundary transitions. The effects of major phenomena such as long-lasting meteorological events may induce strong, non-periodic variance by affecting the real demand peak in ways that GTFS tables cannot detect.

The actionable aim of this report is to test whether the static scheduling of GTFS fails to track the boarding demand stream. This examination is approached from an extrinsic, operational angle where we measure the efficacy of $\mathcal{T}$ to track a non-drifting subset stream of $\mathcal{S}$. We also approach this from an intrinsic, conceptual angle where we measure regime-shifts/concept-drifts strictly in $\mathcal{S}$ and compare them to external covariates, e.g., weather. In this manner, we can rigorously evaluate the efficacy of the static GTFS data, as well as how demand streams behave on their own (to track rush-hour patterns).

We extract the temporal boundaries from $\mathcal{S}(t)$ using an an adaptive threshold, and complementarily use drift detection tools on our best streaming model. Boundary shifts are evaluated using heuristic/naive official windows and the exact GTFS official windows from `stop_times.txt`. Moreover, we use exogenous covariates (precipitation, holidays) to evaluate environmental effects on $\mathcal{S}$. We posit that even if the scheduling data were optimally calibrated to a predictable stream, the boarding behavior may display environmental covariate-sensitivity which manifests as concept drifts – since, static tables cannot perform these real-time observations, exposing the limits of batching techniques and the immediate-response requirements of public transport management.

To avoid look-ahead bias and heavy memory footprints, all streams are ran through an online, prequential regression CapyMOA framework.

Working within this infinite stream constraint, our contri-

butions are:

- **Prequential Demand Forecasting Architecture:** An online regression pipeline (utilizing CapyMOA's ARF, FIMTDD, etc., estimators) to model the non-stationary boarding volume. This is evaluated via an interleaved test-then-train (prequential) framework in order to eliminate look-ahead bias, and validated by windowed RMSE and MAE metrics against *naive* baseline estimators (e.g., a Markovian $\hat{y}_t = y_{t-1}$, or yesterday/last-week same interval).
- **Adaptive Peak Detection:** A baseline of boarding rate $\rho(t)$ using Exponentially Weighted Moving Averages (EWMA) over 5-minute passenger aggregation streams. An interval is a "peak" when the smoothed rate crosses an adaptive threshold

$$\tau = \mu + k \cdot \sigma + \delta,$$

  i.e., an *online update* of the baseline mean, variance, and absolute drift δ; locating the exact peak demand boundaries times, *independently* of static scheduling assumptions.
- **Multivariate Drift Tracking:** The CapyMOA ADWIN change-detector to flag when boundary shifts occur (regime shift or concept drift). We directly correlate these shifts with external variables, specifically Open-Meteo historical API —hourly/rain precipitation, temperature for Bahia, Brazil.
- **Operational Phase-Shift Evaluation:** Evaluate the shifts between our empirical stream boundaries and the static `stop_times.txt` scheduled arrival supply table. We aim to expose the limitations of static scheduling and form actionable criteria for public transport capacity management.

2. Related Work

Offline batch methods of data analysis on transit demand are the traditional approach that public transit networks rely on, hence, their corresponding GTFS (stable, long-horizon) tables are poorly equipped for real-time response. Static arrival schedules are vulnerable to repetitive, high-signal, exogenous shocks which noticeably shift the system into new regimes. These challenge the capacity for operators to respond rapidly. Traffic accidents can present similar shifts; however, in terms of stream-mining, they would appear as transient outliers rather than true concept drifts, since most traffic accidents are resolved within a narrow window (e.g., 24 hours). Our study utilizes the Salvador Urban Network for Transport (SUNT) dataset [1].

To circumvent batching limitations, literature has shifted toward online stream processing. Ensembles such as Adaptive Random Forests (ARFs) and Fast Incremental Model Trees with Drift Detection (FIMTDD) have demonstrated noticeable success at maintaining stable and predictive accuracy on streams. Through strict prequential evaluation –testing the current instance before training on it– stream-simulation techniques eliminate look-ahead bias on stationary data.

Moreover, to gauge the efficacy of streaming architectures, we rely on explicit change-detection tools. Adaptive Windowing (ADWIN), dynamically widens or shrinks the memory boundaries over time, depending on the variance of a predictive error rate (ϵ) or the variance of the data (σ_W^2). This algorithm was used in order to track the (regime) shift boundaries of the transit queuing system itself, on top of tracking model decay (e.g., concept drift).

3. Methodology and Architecture

3.1. Exploratory Data Analysis and Baseline Profiling

An Exploratory Data Analysis (EDA) studies the overall properties of the SUNT and GTFS data. This helps to filter the data, document important global behavior, and establish clear categorical splits (e.g., weekday vs. weekend). The SUNT data covers $\sim$ 700,000 daily passengers across 400 bus lines. We use the 10-stop, 5-min. aggregated subset from Hugging Face (`labiaufba/SSA_StopBusTimeSeries_5`), which provides the fine temporal resolution required to resolve peak boundaries at 5-min. resolution. Three parallel CSVs —`boarding, landing, loader`— use a wide format of 5-min timestamp rows and 10 stop IDs as columns, which span 92 calendar days (66 weekdays, 26 weekends). This serves as a practical ground to avoid any computational bottlenecks while still capturing a representative subgraph of the entire network (especially by including major transit stops such as Estação Lapa and Shopping da Bahia).

Ingestion, Streaming & Data Handling. The three CSVs are fetched at run-time via `hf_hub_download()` into `data_cache/`; cached files are reused on subsequent runs. This is fully reproducible via `python -u process_sunt_v2.py`. These tables are then passed through an $O(1)$ chunked-streaming setup. By consuming the data in lock-step via `pd.read_csv(chunksize=1000)`, the pipeline ensures that $O(\cdot)$ is invariant to the stream length. No CSV is fully mounted to the RAM, thus simulating the required online-learning process. The data is robustly guarded by ensuring timestamp mismatches return `ValueError` and by checking row counts per chunk, with any mismatch returning a `RunetimeError` immediately to avoid desyncing scenarios. Any missing 5-min. slots are filled using a per-(HH:MM, stop) EWMA ($\alpha = 0.05$) updated exclusively from real observations, ensuring the imputation process remains strictly leakage-free. Imputed records are flagged with `data_valid=0` to allow for independent downstream handling.

Downstream CapyMOA Interface. CapyMOA-optimized wrappers are used as a suitable interface for online learning. Per-stop streaming generators are called via `get_stop_stream(stop_id)` and `get_all_stop_streams()` yield "`RegressionInstance`" objects with `y=boardings`.

Online Learning Compliance. Chunked reads, incremental real-fed EWMA, per-stop generators, and zero batch fit guarantee a true prequential (test-then-train) loop with no data leakage.

Table 1: Streaming Features and Target Variables with CapyMOA.

ID	Feature	Type/Range	Role
0	`hour`	float [0–23]	Time-of-day signal
1	`minute`	float $\{0, 5, \ldots, 55\}$	Intra-hour resolution
2	`day_of_week`	float [0=Mon, …, 6=Sun]	Weekday/Weekend flag
3	`alightings`	float ≥ 0	Covariate
4	`occupancy_load`	float ≥ 0	In-vehicle load
5	`data_valid`	{1.0=real, 0.0=imp}	Imputation flag
6	`boardings`	float ≥ 0	Regression target $y^\star$

3.2. Data Integration and Streaming Features

We then examine how exogenous environmental covariates and historical features better expose the behavior of the passenger demand stream. This setup ensures that online regressors distinguish between the expected weekday-weekend variance and the genuine concept drifts that are derived from external factors.

Exogenous Covariate Fusion. We fetch hourly weather (precipitation + temp.) data from the Open-Meteo API (`archive-api.open-meteo.com`) and organize it into `features_stream.csv`. To avoid look-ahead bias, the hourly data is joined to the 5-min. boarding data via *forward-fill* and aligned to local Salvador time (UTC-3). We use 26 streaming features (including the stop IDs) to map out the augmented data. Some of these include: rain flags (booleans, intensities), precipitation, temperature, `minute_of_hour` (12 5-min. intervals), `cos_hour` (continuous cyclic encoding recognizing 23:55~00:05), etc. Additionally, `is_holiday, holiday_name` flags are used to map official Brazilian national holidays within the stream window (no local Bahia holidays occur within the data's lifespan).

Feature Engineering: All features computed incrementally using only past data; no look-ahead. Lag features: 5, 10, 15, 30, 60 min. Rolling windows: 30 min (6 intervals) and 60 min (12 intervals).

Rain threshold: is_raining = True when `precipitation_mm` > 0.1 (standard meteorological trace-rain threshold). Intensity bins: none (0-0.1), light (0.1-2.5), moderate (2.5-7.6), heavy (>7.6 mm/hr).

Reproducibility: The script `role_b_features.py` is fully reproducible. Weather data is cached after first fetch. The Hugging Face boarding data is cached locally. Re-running the script produces identical output.

3.3. Online Demand Modeling

The following implementation into `pipeline.py` reads the forward-filled data strictly from Section 3.2. We keep `boardings` strictly as the target, and `hour_of_day, sin_hour, boarding_lag, precipitation_mm`, etc. as the features — with all lag/rolling features using only past observations (updated after the prediction step) to ensure prequential replay.

Model Selection and Baselines. Four models are used: Adaptive Random Forest Regressor (ARF: ensemble of decision trees tracking variance that prune themselves at moments of drift), Fast Incremental Model Tree with Drift Detection (FIMTDD: adjusts tree nodes based on error ϵ), Passive-Aggressive Regressor (PA: updates weights drastically if ϵ surpasses a threshold, remains inactive if $\epsilon \sim 0$), and the Stochastic Gradient Descent (SGD) Regressor.

As a rigorous benchmark to test these algorithms against the autocorrelation common in short-term demand, we evaluate using naive estimators that ignore exogenous features:

- `baseline_previous_interval`: $\hat{y}_t = y_{t-1}$ (Naive, Markov-like predictor).
- `baseline_rolling_mean_6`: $\hat{y}_t = \frac{1}{6}\sum_{i=1}^{6} y_{t-i}$ (smooth average of the last 30 minutes).
- `baseline_last_week_same_interval`: $\hat{y}_t = y_{t-2016}$ (2016 5-min. intervals = 1 week).
- `baseline_official_window_mean`: separate running means for official peak / off-peak.

Model capacity will be measured by the ability to capture the complex exogenous activity from `features_stream.csv`, instead of memorizing the isolated boarding activity, by contrasting the CapyMOA regressors with these naive estimators. To gauge both model decay and the overall accuracy of the models, the pipeline tracks both windowed and cumulative MAE and RMSE.

Reproducibility:
`python -m src.pipeline` (ARF vs baselines, all 10 stops)
`python -m src.pipeline --models arf fimtdd pa sgd` (full all-model run)
`python -m src.pipeline --max-instances 2000` (quick debug run)

3.4. Adaptive Boundary Extraction

We frame peak-hour boundary detection as a streaming change-detection problem and approach it from two complementary angles, both using the no-look-ahead feature stream of 3.2. (`features_stream.csv`).

Framing 1 – data-driven boundaries (signal level). To extract the demand-side boundaries, we stream the 5-min. boarding counts for each stop through an online adaptive-threshold detector: an EWMA keeps a running off-peak mean μ and variance σ^2, and an interval is flagged "peak" when its boarding rate exceeds a dynamic threshold τ_t,

$$\tau_t = \mu_{t-1} + k \cdot \sigma_{t-1} + \delta,$$

– where δ is an absolute drift – and is sustained for ≥ 20 min inside a morning or evening search-window. The first sustained crossing each day is the data-driven peak *start*. The resulting per-day boundary series is then monitored with CapyMOA ADWIN to test for temporal drift.

Framing 2 — concept drift on the online model (model level). Stream mining in this setting should distinguish between *structural regime shift* (deterministic shifts caused by expected, cyclic covariates like weekdays-weekends) and *classical concept drift* (causing model decay over time). For the latter, we take the online Adaptive Random Forest regressor from 3.3. and feed its prequential error into ADWIN, both per-instance and as a per-day RMSE series, to detect when the model's error regime shifts (classic concept drift). Everything is single-pass / incremental, satisfying the streaming constraint.

3.5. Supply-Side Schedule Alignment

Rather than evaluating on the naive windowing, this method uses the real GTFS data. We filter the `stop_times.txt` (~ 88MB) as well as `trips.txt`, `calendar.txt` to our subset of 10 stops, with weekday/weekend flags and wrap mod 1440 (mins/day). For each stop, scheduled arrival times are mapped into 5-minute bins and flagged by day-type (weekday-weekend) to build a scheduled arrival profile. The scheduled peak start is then extracted using an equivalent adaptive-threshold detector as 3.4.–*Framing 1*. This allows us to directly compare the scheduled-onset with the demand-onset. Files: `peaks.py`, `drift.py`, `pipeline.py`.

4. Results

Exploratory Data Analysis. EDA characterizes each stop by its mean 5-min. boarding count as a function of hour of day, stratified into weekday and weekend profiles, accumulated online across the stream. Imputed records (data_valid=0) are excluded from profile accumulation.

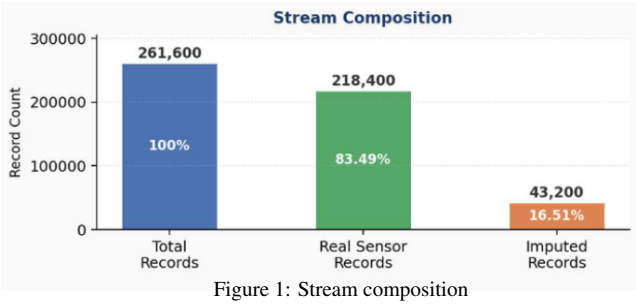


Figure 1: Stream composition

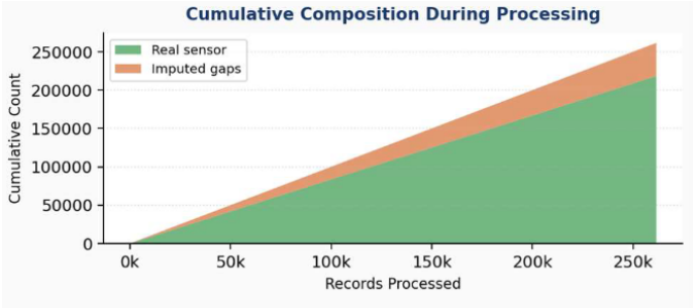


Figure 2: Cumulative processing. Constant 16.51% gap rate across all five 50k checkpoints confirms uniform temporal distribution of missing slots — validating the EWMA scheme.

Representative Stop Profiles.

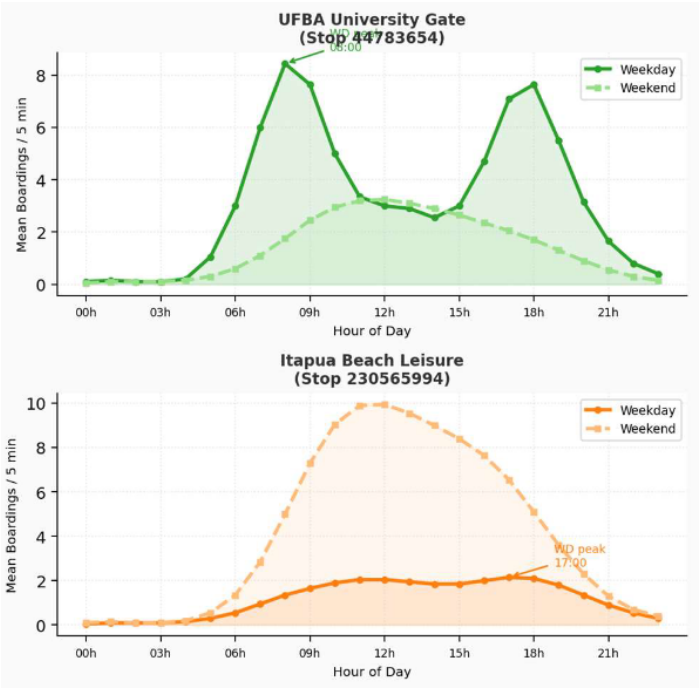
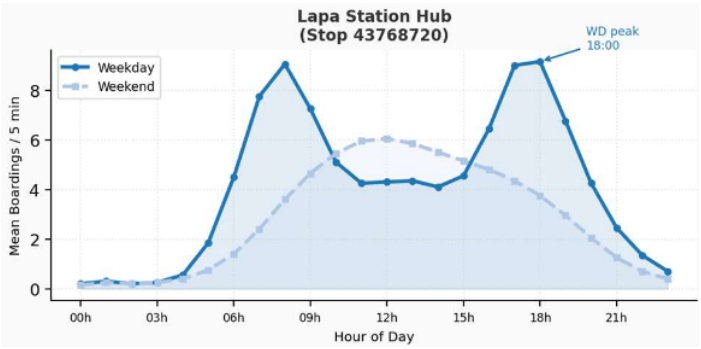


Figure 3: These and the following figure — mean 5-min boarding count by hour of day for 3 representative stops. Weekday (solid) vs Weekend (dashed). Peak annotations mark dominant weekday boarding hour. Source: `process_sunt_v2.py` EDA output, Mar–May 2024.



Three Archetypal Stops.

- **UFBA Uni. Gate (44783654) — Commuter Hub:** Weekday: near-zero 05:00 → sharp ramp → peak ~9.1 boardings/5 min at 08:00 → evening return 17:00–18:00. Weekend collapses to flat plateau (peak ≈3.3). Weekday-to-weekend ratio ~6.5× — highest in dataset. Official 06:00–09:00 window already past peak by 09:00.
- **Lapa Station Hub (43768720) — Interchange Twin Peaks:** Twin weekday peaks: 07:00 (≈9.4) and 17:00–18:00 (≈10.2 — dataset maximum). Weekend elevated (peak ≈6.1) due to interchange role. Highest-priority stop for drift detection: a 15- min boundary shift misallocates capacity across all connecting routes.
- **Itapua Beach Leisure (230565994) — Weekend Inversion:** Weekday near-flat (peak ≈2.2). Weekend sustained midday plateau ≈9.7–10.1 between 10:00–14:00 — a 4.6× inversion. Directly contradicts fixed official schedule. On a Sunday in March 2024, Itapua's busiest hour is 12:00, not 08:00.

Observations.

- **Real boundaries diverge systematically from official schedule.** Fixed 06:00–09:00/17:00–20:00 windows misclassify ≥7 of 10 stops simultaneously. UFBA peaks before 09:00; Itapua has no weekday peak; Castro Alves exceeds 20:00 on weekends.
- **Day-type stratification is structurally necessary.** Commuter stops lose 60–80% of weekday peak volume on weekends. A detector on an undifferentiated stream conflates day-type effects with concept drift — per-day-type models are essential, not optional.
- **Uniform gap distribution validates the EWMA scheme.** 16.51% imputation rate is constant across all five 50,000-record checkpoints (8,160 gaps/block). The EWMA baseline is well-conditioned – real observations for every HH:MM bucket precede any imputation call.
- **Per-stop adaptive thresholds are non-negotiable.** Four archetypes emerge – commuter hubs (UFBA, Bus Station), interchange nodes (Lapa), leisure/event stops (Itapua, Barradao, Fonte Nova), evening/cultural (Castro Alves, Lacerda). A single global threshold is actively harmful.
- **Drift test candidates from EDA:** Good Friday (29 Mar), Tiradentes (21 Apr), Labour Day (1 May) in the window offer testable one- day depressions at commuter stops. Open-Meteo weather data should correlate rainfall with Itapua's weekend plateau shift.

Priority recommendation for downstream: Begin prequential experiments on UFBA (44783654) and Lapa (43768720): strongest weekday ramps, cleanest weekday/weekend separation, stable 16.51% imputation rate. Scale to all 10 stops once the boundary detector is validated on these two.

4.1. Results from 3.3. and 3.4.

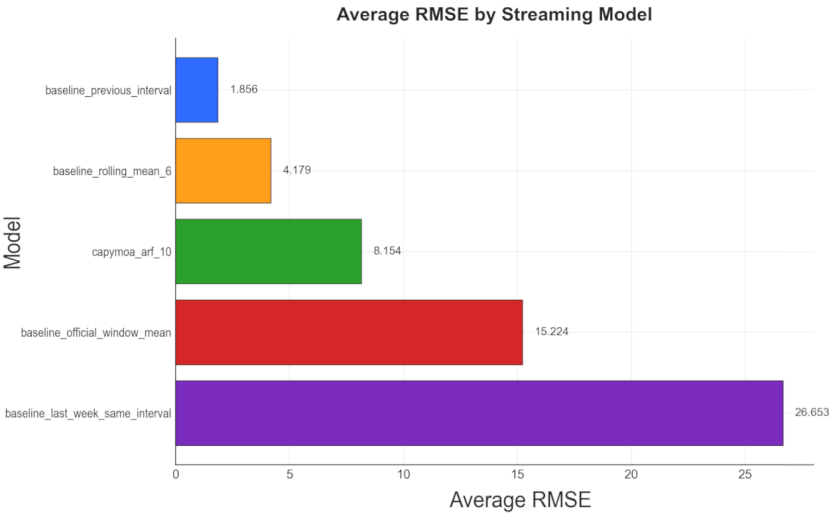


Figure 4: Lower RMSE indicates better boarding-demand prediction, very similar hierarchy for MAE.

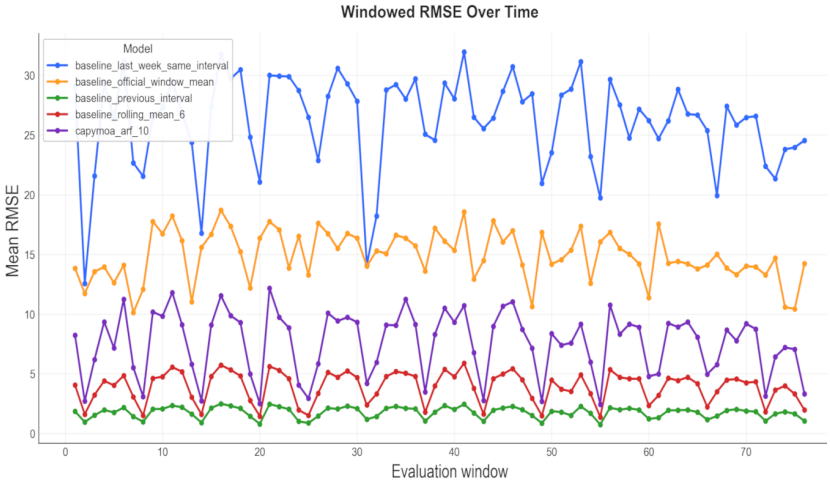


Figure 5: Windowed RMSE reveals whether prediction error increases during changing demand patterns, almost exact behavior as MAE but higher by a constant of 5 on yellow and blue.

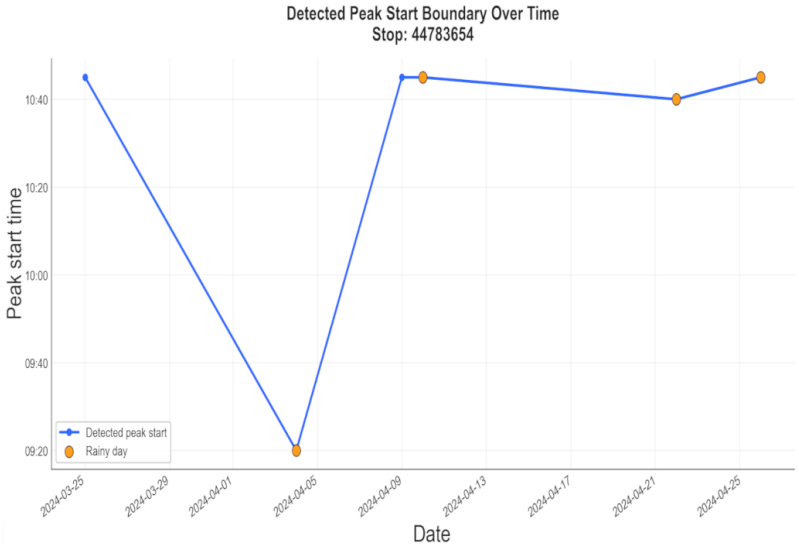


Figure 6: Peak start time is shown as clock time. Rainy days and holidays are highlighted when available.

4.2. Main Results Summary

`baseline_previous_interval` is the strongest measure overall. Short-term 5-minute boarding demand is highly autocorrelated, so the most recent count is a very strong predictor of the next interval – as expected of Markovian-like predictors on stable streams. Among CapyMOA models, **PA is best**, and it *beats* the previous-interval baseline at the busier, more variable **Lapa Station** (MAE 6.65 vs 7.22) – evidence that adaptive online models help at high-variance hubs. **ARF** runs correctly and gives a valid comparison but does not beat the strongest simple baselines on this short horizon. **SGDRegressor** and **FIMTDD** are **unstable** (SGD diverges to ~1e11 errors, FIMTDD shows exploding RMSE). We consider these to be unreliable without further tuning.

The regression shows that demand is predictable and hence that proper baselines were used – since we should expect that public transport demand, in most circumstances, follows a smooth **trend**, even if that trend is Markovian, as our previous-interval baseline suggests.

Table 2: ADWIN concept drift tracking ARF error with per instance or daily measures. The absence of Daily RMSE drift confirms that high-freq. variations are predictive instead of signifying real model decay.

Stop Name	ARF RMSE	Detects/Inst.	Drift-Days/Inst.	RMSE Drifts/Day
Lapa Station	67.104	186.0	86.0	0.0
Salvador Bus Station	12.482	136.0	79.0	0.0
Lacerda Elevator	4.263	119.0	77.0	0.0
Shopping Mall	1.629	126.0	80.0	0.0
Castro Alves Theater	1.281	58.0	35.0	0.0

Table 3: Data-Driven peaks compared to our idealized official hours.

Stop	Board-Peak(signal)	Naive Sched.	Weekend Shift (Var.)
Lapa Station (AM)	**06:09**	06:00	+11 min
Salvador Bus Station (PM)	**17:05**	17:00	+48 min
Shopping Mall (PM)	**15:57**	17:00	**+193 min**
Castro Alves Theater (PM)	17:20	17:00	+89 min
Lacerda Elevator (PM)	17:06	17:00	+72 min

Morning rush is a clean signal only at the origin hub Lapa (06:09, stable). Biggest, most consistent effect: the weekday→weekend shift (Mall's evening peak slides ~3 h later at weekends).

Drift: ADWIN finds **0 temporal drift** on the boundary series (Framing 1) *and* **0 drift** on the model's per-day RMSE (Framing 2). Per-inst. ADWIN on error fires almost daily – due to strong daily seasonality, a methodological caveat, not real drift. Numbers: `outputs/role_d_concept_drift.csv`. Furthermore, the results on `stop_times.txt` show:

Table 4: Misalignment gaps between the real GTFS scheduled service peaks and boarding demand peaks during the weekday morning (AM) window.

Stop	Demand Peak	Scheduled Peak	Gap
Lapa Station	06:09	05:50	+19 min
Salvador Bus Station	06:20	05:45	+35 min
Ferry terminal	06:46	06:00	+46 min
Castro Alves Theater	07:21	06:30	+51 min
Lacerda Elevator	07:27	06:25	+63 min

PM gaps are unreliable. Afternoon service is a long plateau, so "first threshold crossing" fires near the window open (~15:00) and inflates PM gaps (+100 to +250). the AM comparison is the trustworthy one because both demand and supply have a sharp morning ramp. Across commuter/hub stops the timetable *front-loads* morning service roughly *20-60 minutes before* real boarding demand peaks.

UFBA AM demand = 10:30 was classified as a detector artifact, making the +275-minute gap uninformative under standard commuter assumptions. Alternatively, this signal may be caused by a structural covariate based on course-schedules. The latter would be explained by the transit authority deploying maximum capacity $\kappa(t)$ early in the day if they assume UFBA is a common commute-stop, but while also lowering the capacity before the real demand $\rho(t)$ reaches its maximum – this would verify the schedule's susceptibility to high demand bottlenecks. Several event/leisure stops (Itapua, Arena, Barradao) have *no clear demand peak*, so the gap is undefined – itself a finding (these stops are event-driven, not commuter-driven). The local `scripts/04_schedule_alignment.py` reproduces the same logic on Role 3.2 feature table (`process_sunt` gap-imputed), so small differences are expected.

5. Discussion and Actionable Findings

Table 3: The 3-month window (Mar–May 2024) shows no slow temporal drift, so this project's contribution is identifying the *structural* boundary differences and the gap between official windows and the true, stop-specific peaks. Limitations: it is a 3-month, 10-stop subset (no carnival); rain is a weak covariate because Mar–May is Salvador's wet season (≈74% of days rainy).

Destination stops (e.g. UFBA) carry their morning signal in *alighting* (landing/arrival), not boarding, so a morning analysis there would use the alighting series. However, in the context of urban planning, the transit director focuses on making a rigorous schedule instead of continually managing the network flow itself. Hence, the schedule data is a much better candidate for actionable analyses, as it is the only variable the director can act on. With this we can prepare the schedule to handle demand drift or unpredictable high-variance scenarios, suggesting the need for covariate analysis as was done here. Moreover, in Table 4. we observe that the gaps of the EWMA scheduled peaks vs. boarding peaks actually appear with the demand always coming after the scheduled peak. This verifies the necessary deployment of transit capacity $\kappa(t)$ at any point *before* queue formation, even if by overestimation. As explained above, the divergence highlighted by the gaps suggests a misalignment. An actionable route would be to shift the first reinforcement runs ~20-40 min later at these stops so buses arrive *with* demand, not ahead of it.

6. Conclusion and Future Work

In this report, public transport schedule data optimized by batch methods was analyzed with an infinite stream-mining approach. To measure the operational divergences in the SUNT data, we used online, prequential regressors and adaptive boundary extractors. These quantified the shifts between the real boarding demand $\mathcal{S}$ and the static schedules $\mathcal{T}$, as well as the shifts inherent to $\mathcal{S}$ itself. Our results prove that in high-variance network nodes, static peak-hour assumptions fail to track boarding demand and cause early capacity $\kappa(t)$ overload. Resulting in the unmitigated and **costly** growth of $Q(t)$ when $\kappa(t)$ is poorly managed.

Management of transit networks rely on $\mathcal{T}$ as a control. The behavior of the gaps/misalignments show that $\mathcal{T}$ can at times be "excessively prepared" for $\mathcal{S}$; high-variance scenarios suggest room for optimization and adjustments instead of global fixes. Moreover, while covariates like precipitation had a weak signal during our constant wet-season regime, the variance magnitudes inferred from Table 2. verify the necessity to introduce exogenous features to our analysis.

Future work should scale this methodology to the entire stop network across an entire year. As well as expand the feature-space to consider high-dimensional regime boundaries (Carnivals + Holidays + Seasons, etc.); allowing us to gauge the idealized optimization $\kappa(t) \approx \rho(t)$.

CRediT Authorship Contribution Statement

Aryan: Data curation, Formal analysis, Software.
Eugene: Conceptualization, Project administration, Writing – Original Draft Preparation, Writing – review & editing.
Hans: Formal Analysis, Methodology, Software, Validation.
Necmi: Formal Analysis, Methodology, Software, Project Administration, Validation.
Saurabh: Formal analysis, Software, Validation, Visualization.
Van: Data curation, Investigation, Software, Visualization.

All members collaborated on the report.

References

[1] M. V. Ferreira, M. Souza, T. N. Rios, I. F. C. Fernandes, J. Nery, J. Gama, A. Bifet, R. A. Rios, *Salvador Urban Network Transportation (SUNT): A Landmark Spatiotemporal Dataset for Public Transportation*, *Scientific Data* **12** (2025), no. 1, 1320. doi:10.1038/s41597-025-05674-6.